MSM — Database Architecture

Application: My Machine Shop Manager (MSM) — CNC / machine-shop management (single-tenant SaaS)

Backend: Supabase (PostgreSQL) — accessed **Supabase-direct** from the client with the anon key; all rule-bearing writes go through Postgres RPCs.

Source of truth: `docs/supabase-schema.sql` (base DDL) + `docs/supabase-approval-policy.sql` (security gate) + `supabase/migrations/0001..0029_*.sql` (applied in order).

> This document is generated from the SQL migrations. It is the authoritative map of tables, views, functions, enums, security policies, and the numbering/audit machinery.

1. Design conventions

Convention	Detail
Primary keys	`TEXT`, app-generated string IDs (e.g. `cmp_ab12cd`, `job_...`). Not UUIDs.
Document numbers	Human-facing, gapless-ish sequences (`INV-0007`, `DC-0003`) issued by `next_seq()` from the `doc_counters` table.
Money	`numeric(14,2)`. **Quantities** `numeric(14,3)`. **Tax %** `numeric(6,3)`.
Timestamps	`created_at` / `updated_at` `timestamptz default now()` on nearly every table.
Enums	Core sales/production use real PG `enum` types; newer modules (HRM, Accounting, GST, Tool Room) use `text` + `CHECK` constraints instead.
Company scoping	Many tables carry a nullable `company_id`. `NULL` conventionally means "shop-owned / shared / all companies" (represented as `'*'` in unique indexes via `coalesce(company_id,'*')`).
Rule enforcement	Business rules live in `SECURITY DEFINER` / `SECURITY INVOKER` RPCs and triggers, so they hold regardless of client. Direct table writes are the exception, not the rule.
Idempotency	Stock dispatch and tool moves use unique keys (`reference_type/reference_id/material_id`, `ref_key`) so replays don't double-post.

2. Security model

MSM is single-tenant but has a **hard approval boundary** enforced entirely in the database.

```
auth.jwt() email
      │
      ├─ is_super_admin()   → email ∈ SUPER_ADMIN_EMAILS (admin@sreebalajiindustries.com)
      │
      └─ is_app_approved()  → is_super_admin() OR email ∈ approved_users
                                        │
                                        ▼
                    RLS "approved_all" policy on every business table
```

- `approved_users` — approval registry. RLS enabled with **no policies** → no client can read/write it directly. Only `SECURITY DEFINER` functions (running as table owner) touch it. This is what makes approval tamper-proof: a pending user with a valid token gets **zero** data access.
- `is_super_admin()` — `SECURITY DEFINER`, matches JWT email against a hardcoded list (must stay in sync with `SUPER_ADMIN_EMAILS` in `src/features/auth/auth.tsx`).
- `is_app_approved()` — the gate every data-table RLS policy calls.
- `set_user_approval(email, approved)` — approve (insert) / revoke (delete). No-ops unless the caller is a super admin (checked inside).
- `app_state` stays open to all authenticated users on purpose (sign-up records the pending profile; login reads applicant status). It holds no business records and cannot bypass the gate.

RLS evolution: Early tables used a permissive `auth_all` policy (`to authenticated using(true)`). The approval-policy migration replaced it with `approved_all` (`using(is_app_approved())`) on all business tables. HRM/Finance originally had fine-grained per-permission RLS

(`ACCOUNTS_VIEW` , `BANK_IMPORT` , etc.); migrations **0026** (finance) and **0027** (HRM) **collapsed those to a single** `approved_all` **(`is_app_approved()`) policy per table** — permission granularity now lives in the RPC layer and the UI, not RLS. Ledger/audit tables (`tool_transactions` , `hr_audit_log` , `journal_lines`) are read-gated but write-only through their `SECURITY DEFINER` functions.

3. Module map

87 tables + 7 views across 14 functional modules:

#	Module	Key tables
A	**Master data**	`companies` , `materials` , `products` , `vendors`
B	**Job orders & production**	`job_orders` , `production_events`
C	**Inventory / stock**	`material_receipts` , `material_issues` , `stock_adjustments` , `stock_transfers` , `own_material_purchases`
D	**Sales**	`invoices` , `invoice_lines` , `payments` , `delivery_challans`
E	**Subcontracting**	`subcontract_orders` , `subcontract_docs`
F	**Expenses**	`expenses`
G	**CRM**	`contact_messages`
H	**Auth & approval**	`app_state` , `approved_users`
I	**HRM foundation (RBAC)**	`hr_permissions` , `hr_roles` , `hr_role_permissions` , `hr_user_roles` , `hr_settings` , `notifications` , `hr_audit_log`
J	**HRM core**	`employees` , `departments` , `designations` , `shifts` , `shift_assignments` , `holidays` , `attendance` , `leave_types` , `leave_balances` , `leave_applications` , `employee_status_history`
K	**HRM payroll & lifecycle**	`salary_*` , `employee_salary` , `payroll_*` , `employee_documents` , `employee_assets` , `asset_assignments` , `employee_advances` , `advance_repayments` , `expense_claims` , recruitment (`job_openings` , `candidates` , `interview_rounds` , `job_offers`), performance, training
L	**Accounting (double-entry)**	`chart_of_accounts` , `fiscal_years` , `accounting_periods` , `journals` , `journal_lines` , `bank_accounts`
M	**Banking / import**	`bank_statement_files` , `bank_transactions` , `bank_txn_rules` , `party_aliases`
N	**GST & compliance**	`gst_registrations` , `gst_tax_rates` , `hsn_codes` , `gst_return_periods` , `einvoice_records` , `eway_bills`
O	**Tool Room**	`tool_categories` , `tools` , `tool_transactions` , `tool_reservations` , `tool_maintenance` , `tool_calibrations`
—	**Infra**	`doc_counters` (numbering), `audit_log` (legacy audit)

4. Enums (real PostgreSQL types)

Enum	Values
`job_status`	Draft, Pending, In Progress, On Hold, Completed, Delivered, Cancelled
`job_priority`	Low, Normal, High, Urgent
`invoice_status`	Draft, Unpaid, Partially Paid, Paid, Cancelled
`payment_method`	Cash, Bank Transfer, UPI, Cheque, Other
`owner_type`	Company, Shop

> Newer modules encode their status/kind vocabularies as `text` + `CHECK` (documented per table below) rather than PG enums, to keep migrations additive.

5. Core domain (A–G)

A. Master data

`companies` — customers. `id` PK, `code` unique, `name`, contact fields, `gstin`, `billing_address`, `active`, `notes`. Seeded with C001 Flowra Global, C002 Vahinie Engineering, C003 Nirmal Pumps, C004 Local.

`materials` — raw materials/stock items. `id` PK, `code` unique, `name`, `type`, `unit`, `default_rate`, `reorder_level`, `active`, `company_id` (nullable → shared/own; added in 0010).

`products` — rate list (machining cost per part). `id` PK, `code` unique, `name`, `rate`, `unit`, `hsn`, `active`.

`vendors` (0013) — suppliers/subcontractors. `id` PK, `code` unique, `name`, `gstin`, phone/email/address, `active`.

B. Job orders & production

`job_orders` — the central production entity.

`id`, `job_no` (unique) · `company_id` →companies · `customer_po`, `part_name`, `part_number` · `material_id` →materials · `ordered_qty (>0)`, `completed_qty (≥0)`, `rejected_qty` (QC, added via docs SQL) · `rate` · `order_date`, `due_date` · `priority` (job_priority) · `status` (job_status) · lifecycle timestamps `started_at` / `completed_at` / `delivered_at` · `operator`, `notes`.

Indexes: `company_id`, `status`.

`production_events` — append-only status/quantity log per job. `id`, `job_id` →job_orders (cascade), `type`, `from_status` / `to_status` (job_status), `completed_qty`, `note`, `operator`, `at`.

C. Inventory / stock — transaction-sourced balances

Stock is **never stored as a single balance**; it's derived from three signed ledgers plus a per-receipt allocation model.

- `material_receipts` — stock in. `id`, `receipt_no` (unique), `date`, `material_id`, `owner_type` (owner_type enum), `company_id`, `job_id`, `supplier`, `quantity (>0)`, `unit`, `rate`, `batch_no`, `reference`, `notes`.
- `material_issues` — stock out. `id`, `issue_no` (unique), `date`, `material_id`, `job_id` (nullable since 0007), `company_id`, `quantity (>0)`, `unit`, `note`, plus **ledger links** `reference_type` / `reference_id` (0007) and **source allocation** `source_receipt_id` →material_receipts (0015). Unique partial key `(reference_type, reference_id, material_id, coalesce(source_receipt_id,''))` guarantees idempotent dispatch.
- `stock_adjustments` — signed corrections & dispatch reversals. `id`, `adj_no` (unique), `date`, `material_id`, `company_id`, `quantity (<>0, signed)`, `unit`, `reason`, `source_receipt_id` (0015).
- `stock_transfers` (0029) — inter-location moves. `id`, `transfer_no`, `material_id` (cascade), `company_id`, `from_location`, `to_location`, `quantity (>0)`, `unit`, `transfer_date`, `requested_by` / `approved_by`, `status` CHECK(draft|requested|approved|in_transit|completed|cancelled).
- `own_material_purchases` (0009) — shop-owned procurement. Links a purchase to the `material_receipts` row (+qty, shop-owned) and the `expenses` row (cost+GST) it generated, so there are no orphans.

Balance formula: `Σ receipts − Σ issues + Σ adjustments`, optionally scoped by company (`is not distinct from` NULL match for shop stock).

D. Sales

`invoices` — `id`, `invoice_no` (unique), `date`, `company_id`, `billing_address`, `shipping_address` (docs SQL), `reference`, `dc_reference` (docs SQL), `discount`, `tax_percent`, `cgst_percent` / `sgst_percent` (docs SQL), `status` (invoice_status), `notes`.

`invoice_lines` — `id`, `invoice_id` (cascade), `job_id`, `description`, `quantity (>0)`, `rate (≥0)`, `line_no`, and stock-dispatch fields `material_id` + `owner_type` (0011) + `source_receipt_id` (0015). A line with `material_id` set deducts stock; challan-imported lines leave it null.

`payments` — `id`, `payment_no` (unique), `date`, `company_id`, `invoice_id` (nullable → advance), `amount (>0)`, `method` (payment_method), `reference`, `is_advance`, plus `source` / `bank_txn_id` (0023, links to bank import).

`delivery_challans` — `id`, `dc_no` (unique), `date`, `company_id`, `job_id`, `reference`, `vehicle_no`, `lines` (jsonb), `status` (Open/Invoiced/Cancelled), `invoice_id`. A `dc_guard()` trigger blocks deletes/line-edits once a challan is invoiced or dispatched.

E. Subcontracting (0013)

`subcontract_orders` — `id`, `sc_no` (unique), `date`, `vendor_id`, `material_id`, `owner_type`, `company_id`, `job_id`, `process`, `unit`, `sent_qty` / `received_qty` / `rejected_qty`, `status` (Open/Partially Received/Received via `sc_status()`).

`subcontract_docs` — outward/inward document trail. `id`, `doc_no`, `sc_id` (cascade), `direction` (OUT/IN), `doc_kind` (DC/INVOICE), `vendor_ref`, `date`, `quantity (>0)`, `rejected`, `unit`, `amount`, `expense_id` →expenses.

F. Expenses

`expenses` — `id`, `expense_no` (unique), `date`, `category`, `amount (>0)`, `method` (payment_method), `vendor`, `reference`, `company_id`, `job_id`, `notes`, plus `source` / `bank_txn_id` (0023).

G. CRM

`contact_messages` (0016) — public marketing contact form. `id`, `name`, `email`, `phone`, `company`, `message`, `status` (default 'new'), `created_at`. **Only table with an `anon` INSERT policy** (public form submission); read/update/delete are authenticated.

---

6. Auth & approval (H)

`app_state` — settings + sequences singleton (`id='singleton'`, `data` jsonb). Also holds the `users` array (profiles + pending/approved status). Open to all authenticated users.

`approved_users` — the tamper-proof approval registry (see §2). `email` PK, `role`, `approved_by`, `approved_at`.

Sign-up flow: `register_pending_user()` (`SECURITY DEFINER`, callable by `anon`) merges/inserts a `pending` profile into `app_state.data.users` (0017 → 0018 merge-aware). A super admin then calls `set_user_approval()` to insert into `approved_users`.

---

7. HRM (I–K)

I. Foundation — RBAC + platform services (0019)

Table	Purpose
`hr_permissions`	~37 permission keys grouped by `module` (EMPLOYEE_*, LEAVE_*, PAYROLL_*, ATTENDANCE_*, etc.). Later migrations append ACCOUNTS/BANK/GST (0022), TOOLROOM (0028), INVENTORY (0029) keys here too.
`hr_roles`	Roles; 4 seeded system roles: `hr_admin`, `hr_manager`, `manager`, `employee`.
`hr_role_permissions`	Junction (role_id, permission_key) with a `scope` CHECK(self/team/department/company/all).
`hr_user_roles`	Assigns roles to users by email, optionally per company (`company_id` null = all).
`hr_settings`	Singleton jsonb (employee-code format, etc.).
`notifications`	In-app notifications (recipient_email, type, title, body, entity link, is_read).
`hr_audit_log`	Rich audit trail (actor, action, entity, before/after jsonb, ip, meta). Write-only via `hr_log()`.

Permission resolution stack: `hr_current_email()`, `hr_scope_rank()`, `hr_has_permission(key, company?)`, `hr_permission_scope()`, `is_hr_admin()`, `hr_my_access()`. Role management via `hr_assign_role()` / `hr_revoke_role()`. (Note: after 0027, RLS uses `is_app_approved()` broadly; these helpers still drive UI capability and some RPC guards.)

J. Core HR (0020)

- `employees` — the master record: identity, contact, emergency, employment (`employment_type`, `department_id`, `designation_id`, `reporting_manager_id` self-ref, `shift_id`, `status`), confirmation/probation/exit fields, bank/payment details, `statutory` jsonb, `archived_at` soft-delete. `employee_code` immutable & unique per company. Rich index set.
- `departments` (self-ref parent, `head_employee_id`), `designations` (grade, dept link).
- `shifts` (times, grace/late/OT windows, `working_days int[]`, overnight flag) + `shift_assignments` (effective-dated).
- `holidays` (company/regional/optional), `employee_status_history` (status change log).
- **Leave:** `leave_types` (quota, accrual, carry-forward rules) → `leave_balances` (per employee/type/year: opening/accrued/used/pending/adjusted) → `leave_applications` (workflow: draft→submitted→manager_approved→approved/rejected/cancelled).
- `attendance` — daily record (check_in/out, computed regular/overtime minutes, status, source), unique per (employee, date).

K. Payroll & lifecycle (0021)

- **Salary:** `salary_components` (earning/deduction; calc_type fixed / %-of-basic / %-of-gross / formula) → `salary_structures` → `salary_structure_lines`; `employee_salary` (effective-dated CTC + structure).
- **Payroll run:** `payroll_periods` (draft→…→finalized→locked) → `payroll_runs` (totals) → `payroll_records` (per-employee snapshot with `earnings` / `deductions` jsonb).

- **Documents & assets:** `document_types`, `employee_documents` (expiry-tracked, Storage path); `employee_assets`, `asset_assignments`.
- **Advances/claims:** `employee_advances` + `advance_repayments`; `expense_categories` + `expense_claims` (employees can self-raise).
- **Recruitment:** `job_openings` → `candidates` → `interview_rounds` / `job_offers`.
- **Performance:** `performance_cycles` → `performance_reviews` → `performance_goals`.
- **Training:** `training_programs` → `training_sessions` → `employee_training`.

---

8. Accounting — double-entry ledger (L, 0022)

`chart_of_accounts` — tree (self-ref `parent_id`, `is_group`). `type` CHECK(asset|liability|equity|income|expense), `system_key` (canonical anchor: cash, bank, ar, ap, gst_output, gst_input, sales, purchase, salary, round_off …), `gst_relevant`, `opening_balance`. Seeded with a full COA (groups 1000–5000 + leaf accounts).

`fiscal_years` → `accounting_periods` — both with `status` CHECK(open|closed|locked); posting is refused into locked periods.

`journals` (header: `journal_no`, `date`, `period_id`, `narration`, `source`, `source_type` / `source_id`, `status` draft|posted|void) → `journal_lines` (`account_id`, `debit (≥0)`, `credit (≥0)`, `party_type` / `party_id`, `line_no`).

`bank_accounts` — ties a real bank account to its `ledger_account_id` in the COA.

Posting engine (all `SECURITY DEFINER`):

- `acc_system(key, company?)` — resolve a system account id by key.
- `acc_period_for(date, company?)` — resolve the open period for a date.
- `post_journal(company, date, narration, lines jsonb, source…, status)` — validates balance (Σdebit = Σcredit), period lock, positivity; inserts header+lines; returns journal id.
- `void_journal(id, reason?)` — non-destructive reversal (status→void).

---

9. Banking / import (M, 0023 + 0025)

- `bank_statement_files` — uploaded statement (file hash for dedupe, `parser_type` csv/xlsx/pdf, `status` uploaded→parsed→reviewed→posted).
- `bank_transactions` — parsed rows with a full review/matching pipeline: `classification`, `matched_party_*`, `matched_invoice_id`, `matched_ledger_account_id`, `confidence`, `dedupe_hash`, `dup_status` (new/possible_duplicate/duplicate/ignored), `review_status`, `reconciliation_status`, `posting_status`, and `posted_payment_id` / `posted_expense_id` / `posted_journal_id` back-links.
- `bank_txn_rules` — auto-classification rules (match field/op/value, direction, → classification + party + ledger, priority, confidence).
- `party_aliases` — narration-text → party mapping to boost matching.

RPCs: `detect_bank_duplicates(file)` (flags dups, non-destructive); `post_bank_txn(txn, …)` (turns one txn into payment/expense + balanced journal, optional invoice allocation); `post_bank_txn_split(txn, splits jsonb)` (0025 — one txn across multiple counter postings). Guards relaxed to `is_app_approved()` in 0026.

---

10. GST & compliance (N, 0024)

Table	Purpose
`gst_registrations`	GSTIN(s) per company (type, state code, PAN, default flag).
`gst_tax_rates`	Rate slabs (seeded 0/5/12/18/28 with CGST/SGST/IGST/cess split).
`hsn_codes`	HSN/SAC master → tax rate.
`gst_return_periods`	GSTR1 / GSTR3B / GSTR2B period status + summary jsonb.
`einvoice_records`	IRN / signed QR / ack per invoice (pending→generated→cancelled).
`eway_bills`	E-way bills (transport details, items jsonb, validity, status).

Reference tables (`gst_tax_rates`, `hsn_codes`) are readable by any approved user; the rest follow the standard `approved_all` gate.

---

11. Tool Room (O, 0028)

A **transaction-driven** inventory subsystem mirroring the material-stock pattern, but with **buckets** instead of a single balance.

- `tool_categories` (self-ref tree; 8 seeded categories) → `tools` (very wide master: identity, specs, stock levels, locations, life/calibration/maintenance flags, `is_consumable`, `is_serialized`, ...).
- `tool_transactions` — the single ledger. Every movement is a row with `txn_type` (receipt/reserve/release/issue/return_*/consume/transfer/maintenance_*/calibrate_*/scrap/adjust), `qty (>0)`, and a `from_bucket` → `to_bucket` transition across buckets **available / reserved / issued / maintenance / calibration / damaged / scrap / consumed**. `ref_key` unique index gives idempotency.
- `tool_reservations`, `tool_maintenance`, `tool_calibrations` — lifecycle sub-records.

RPCs: `tool_bucket_balance(tool, bucket)`; `tool_can(key)` (auth helper — approved AND super-admin/no-roles-yet/has-permission); `tool_move(...)` — the single atomic ledger writer that derives the required permission + bucket transition from `txn_type`, enforces non-negative `available`, and is idempotent via `ref_key`. All writes to `tool_transactions` go through it (`SECURITY DEFINER`).

---

12. Views (7)

View	Module	What it computes
`material_stock`	Inventory	Overall balance per material (receipts − issues + adjustments).
`material_receipt_stock` (0015)	Inventory	Per-receipt allocation grid: received, dispatched (DC/invoice/other), adjusted, `available`, status.
`inventory_ledger` (0007)	Inventory	Unified receipts+issues+adjustments transaction stream (qty_in/qty_out, txn_type, doc_no, reference).
`invoice_totals`	Sales	Subtotal, tax_amount, total, paid, outstanding per invoice (Cancelled → 0).
`general_ledger` (0022)	Accounting	Posted journal lines joined to accounts (GL reporting).
`trial_balance` (0022)	Accounting	Per-leaf-account debit/credit totals + net balance.
`tool_inventory` (0028)	Tool Room	Per-tool bucket snapshot (available/reserved/issued/... + on_hand, out-of-stock & low-stock flags).

---

13. Numbering system (0001)

`doc_counters` (`key` PK, `value` bigint) is the atomic sequence store.

- `next_seq(key)` — increment & return (consumes a number). Used by every create RPC.
- `peek_seq(key)` — preview next value without consuming (UI previews).

Keys: `job`, `invoice`, `receipt`, `issue`, `adjustment`, `payment`, `expense`, `dc`, `companyCode`, `materialCode`, `productCode`, `journal`, plus module document numbers. Seeded from the legacy `app_state.data.sequences` on migration.

---

14. RPC catalog (rule-bearing writes)

The client never writes rule-bearing tables directly — it calls these. Grouped by module:

Numbering: `next_seq`, `peek_seq`

Approval: `is_super_admin`, `is_app_approved`, `set_user_approval`, `register_pending_user`

Stock: `material_balance`, `create_material_issue`, `create_stock_adjustment`, `receipt_available`, `assert_source_dispatchable`

Jobs: `create_job`, `transition_job`

Sales: `create_invoice`, `create_payment`, `delete_payment`, `set_invoice_status`

Challans: `create_challan_with_dispatch`, `update_challan_full`, `update_challan_quantities`, `cancel_challan`, `reopen_challan`, `dc_guard` (trigger)

Purchasing/subcontract: `create_own_material_purchase`, `sc_status`, `create_subcontract_dispatch`, `create_subcontract_return`

HRM: `hr_current_email`, `hr_current_employee_id`, `hr_scope_rank`, `hr_has_permission`, `hr_permission_scope`, `is_hr_admin`, `hr_my_access`, `hr_log`, `hr_notify`, `hr_assign_role`, `hr_revoke_role`, `hr_next_employee_code`, `hr_apply_leave`, `hr_decide_leave`, `hr_run_payroll`, `hr_finalize_payroll`

Accounting: `acc_system`, `acc_period_for`, `post_journal`, `void_journal`

Banking: `detect_bank_duplicates`, `post_bank_txn`, `post_bank_txn_split`

Tool Room: `tool_bucket_balance`, `tool_can`, `tool_move`

> Several RPCs were rewritten across migrations (`create_invoice` 0004→0011→0015, `set_invoice_status`, `create_challan_with_dispatch`, `update_challan_full`, `dc_guard`, `register_pending_user` 0017→0018). The latest version wins; earlier ones are historical.

---

15. Migration history

Range	Theme
base + docs SQL	Core schema, approval gate, invoice CGST/SGST columns, job `rejected_qty`
0001	Document numbering (`doc_counters`, `next_seq` / `peek_seq`)
0002–0006	Stock, job, invoice/payment, challan RPCs; delete-payment
0007–0008	Inventory ledger links + view; challan dispatch (stock-out on DC)
0009	Own-material purchase (receipt+expense atomic)
0010–0012	Material company scope; invoice→stock dispatch; challan qty edits
0013	Supply chain (vendors, subcontracting)
0014–0015	Full challan edit; **per-receipt source allocation** (the big inventory refactor)
0016	CRM contact messages (public form)
0017–0018	Approval-gated sign-up (`register_pending_user`)
0019–0021	**HRM** (foundation/RBAC, core, payroll & lifecycle)
0022–0025	**Accounting** (double-entry), **bank import**, **GST**, bank split
0026–0027	RLS alignment — collapse fine-grained finance/HRM policies to `is_app_approved()`
0028	**Tool Room** (bucketed transaction ledger)
0029	**Inventory module** (`stock_transfers` + INVENTORY_* permissions)

---

16. Relationship highlights (how it ties together)

```
companies ─┬─< job_orders ─< production_events
           │        └─< material_issues / invoice_lines / delivery_challans
           ├─< invoices ─< invoice_lines ─(material_id)─> materials
           │        └─< payments        └─(source_receipt_id)─> material_receipts
           ├─< delivery_challans ─(invoice_id)─> invoices
           ├─< material_receipts / material_issues / stock_adjustments / stock_transfers
           ├─< subcontract_orders ─< subcontract_docs ─> expenses
           ├─< employees ─┬─< attendance / leave_applications / employee_salary / payroll_records
           │              ├─> departments ─> designations ─> shifts
           │              └─< documents / assets / advances / claims / reviews / training
           ├─< chart_of_accounts ─< journal_lines >─ journals ─> accounting_periods ─> fiscal_years
           │        └ bank_accounts ─< bank_transactions >─ bank_statement_files
           └─< gst_registrations / einvoice_records / eway_bills ─> invoices

tools ─> tool_categories;  tools ─< tool_transactions (buckets) / reservations / maintenance / calibrations
approved_users  ⟵ is_app_approved() ⟵ every RLS "approved_all" policy
```

Generated from the SQL migrations in `supabase/migrations/` and `docs/`. Regenerate after adding new migrations.